

Experiment no -1

To study supervised/unsupervised/reinforcement learning approach.
Using
Classify emails as spam or not spam

Machine learning is the branch of Artificial Intelligence that focuses on developing models and algorithms that let computers learn from data and improve from previous experience without being explicitly programmed for every task. In simple words, ML teaches the systems to think and understand like humans by learning from the data.

In this article, we will explore the various types of machine learning algorithms that are important for future requirements. Machine learning is generally a training system to learn from past experiences and improve performance over time. Machine learning helps to predict massive amounts of data. It helps to deliver fast and accurate results to get profitable opportunities.

Types of Machine Learning

There are several types of machine learning, each with special characteristics and applications. Some of the main types of machine learning algorithms are as follows:

- **Supervised Machine Learning**
- **Unsupervised Machine Learning**
- **Reinforcement Learning**

Feature	Supervised Learning	Unsupervised Learning	Reinforcement Learning
Data Labeling	Requires labeled data	No labels required	Uses rewards and penalties
Objective	Predict outputs	Identify patterns	Optimize decision-making
Common Algorithms	Regression, SVMs, Neural Networks	Clustering, PCA, Autoencoders	Q-Learning, DQN, Policy Gradient
Applications	Healthcare, Finance, Marketing	Cybersecurity, Retail, NLP	Robotics, Gaming, Finance

```

import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
data=pd.read_csv("C:/Users/Admin/Desktop/AIML_SE/dataset/spam.csv",encoding='latin-1')
data.head()
data.shape
data.columns
data.drop(columns=['Unnamed: 2','Unnamed: 3','Unnamed: 4'],axis=1,inplace=True)
data.head()
data.columns=['label','message']
data.head()
#Convert labels into numbers:
data['label']=data['label'].map({'ham':0,'spam':1})
X = data['message']
y = data['label']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
texttonum=TfidfVectorizer(min_df=1,stop_words='english',lowercase=True)

X_train_vec = texttonum.fit_transform(X_train)
X_test_vec = texttonum.transform(X_test)
model=LogisticRegression()
model.fit(X_train_vec,y_train)
y_pred=model.predict(X_test_vec)
print("Accuracy:", accuracy_score(y_test, y_pred))
email = ["Congratulations! You won a lottery of $1000"]

email_vec = texttonum.transform(email)
result = model.predict(email_vec)

if result[0] == 1:
    print("Spam Email")
else:
    print("Not Spam")

```

OUTPUT:

Accuracy: 0.9524663677130045

Not Spam

Conclusion: in this way with the help email spam non spam data we learned supervised learning algorithm

EXPERIMENT NO-2

To acquire, visualize and analyze the data set (from time-domain/frequency-domain/ etc.

Time Domain

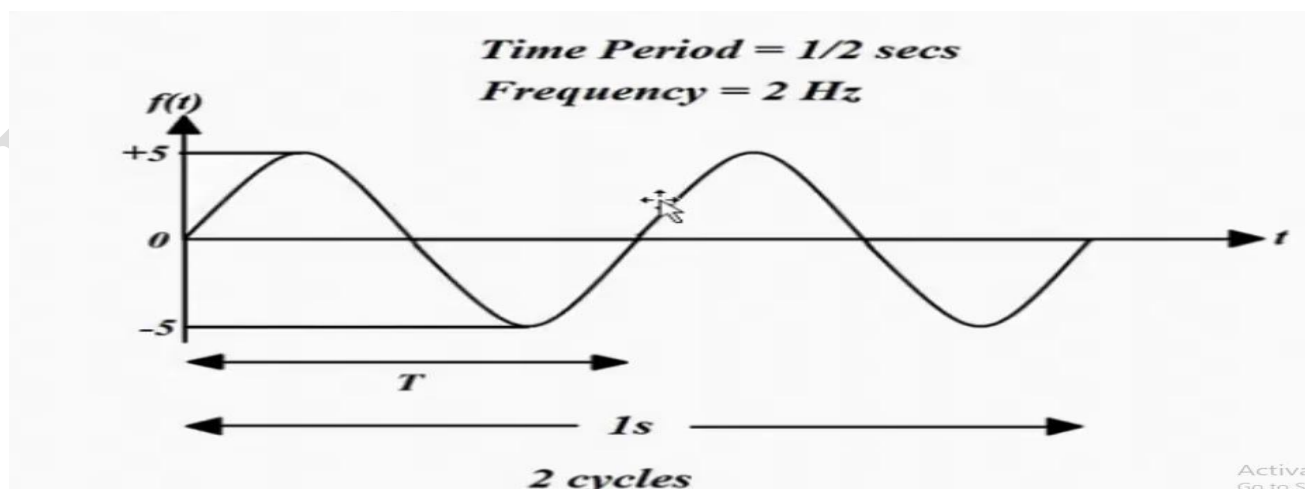
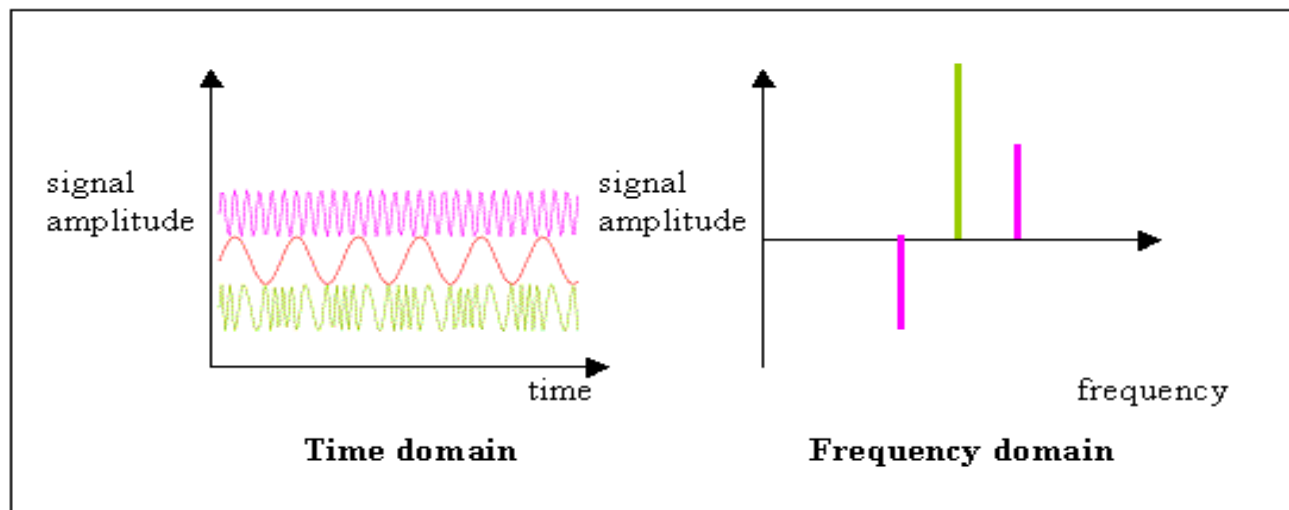
- **Definition:** Analysis of signals with respect to time, showing how amplitude changes over a span of time.
- In the time domain, a signal is represented as amplitude vs time.

Example:

- ECG signal
- Audio waveform
- Stock price over time

Frequency Domain

- **Definition:** Analysis of signals with respect to frequency, showing how much of the signal lies within each frequency band.
- In the frequency domain, the same signal is represented as amplitude vs frequency.
- Audio signal



- The conversion from time domain → frequency domain is done using the Fourier Transform.

6 Why It Is Important (ML / Data Science)

Used in:

- Speech recognition
- Image processing
- Signal denoising
- Feature extraction
- EEG/ECG analysis

Example:

Audio → convert to frequency → train ML model.

Time Domain → Frequency Domain

This is a fundamental concept in signal processing and ML (especially for audio, vibration, sensors).

- In AI & Machine Learning (AIML), converting a signal from **time domain** → **frequency domain** is commonly done using the **Fast Fourier Transform (FFT)**. This is used in areas like **vibration analysis in mechanical engineering**, **fault detection in machines**, and **signal processing**.

• Feature	• Amplitude	• Magnitude
• Meaning	• Height of signal	• Absolute value of complex number
• Domain	• Time domain	• Frequency domain
• Representation	• Peak value of wave	• Strength of frequency component

◆ 2. Why We Do This Conversion

✓ 1. To Identify Hidden Frequencies

A signal may look complex in time domain, but in frequency domain it becomes simple.

👉 Example:

$$x(t) = \sin(2\pi 50t) + \sin(2\pi 120t)$$

Time domain → messy waveform

Frequency domain → two clear peaks (50 Hz, 120 Hz)

✓ 2. Fault Detection (Very Important in Engineering)

Used in:

- Bearings
- Gearboxes
- Motors

👉 Faults create specific frequency signatures

Example:

- Healthy machine → smooth spectrum
- Faulty bearing → spike at fault frequency

👉 This is why vibration analysis uses FFT.

Code

```
import numpy as np
import matplotlib.pyplot as plt

# Step 1: Create time signal
fs = 1000 # Sampling frequency (Hz)
t = np.linspace(0, 1, fs)

# Signal with 2 frequencies: 50 Hz and 120 Hz
signal = np.sin(2 * np.pi * 50 * t) + 0.5 * np.sin(2 * np.pi * 120 * t)

# Step 2: Apply FFT
fft_values = np.fft.fft(signal)
frequencies = np.fft.fftfreq(len(signal), 1/fs)

# Step 3: Take only positive frequencies
```

```
positive_freqs = frequencies[:len(frequencies)//2]  
magnitude = np.abs(fft_values[:len(fft_values)//2])
```

```
# Step 4: Plot
```

```
plt.figure()
```

```
# Time domain
```

```
plt.subplot(2, 1, 1)
```

```
plt.plot(t, signal)
```

```
plt.title("Time Domain Signal")
```

```
plt.xlabel("Time (s)")
```

```
plt.ylabel("Amplitude")
```

```
# Frequency domain
```

```
plt.subplot(2, 1, 2)
```

```
plt.plot(positive_freqs, magnitude)
```

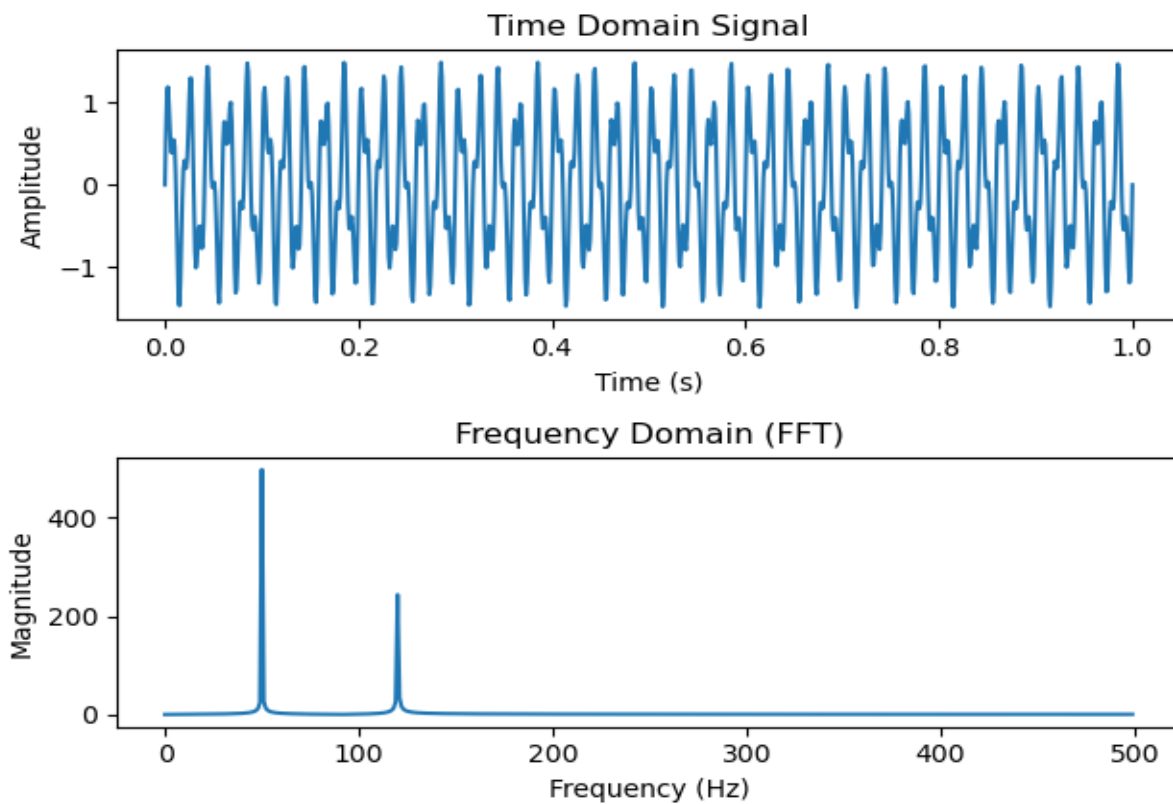
```
plt.title("Frequency Domain (FFT)")
```

```
plt.xlabel("Frequency (Hz)")
```

```
plt.ylabel("Magnitude")
```

```
plt.tight_layout()
```

```
plt.show()
```



conclusion:

- Time domain represents signals as they change over time, while frequency domain shows the signal's frequency components using transforms like FFT (Fast Fourier Transform).
- Converting to frequency domain helps identify patterns such as noise, periodicity, and dominant frequencies.
- In Python, this transformation is commonly done using `numpy.fft` or `scipy.fft`.
- Overall, frequency domain analysis makes signal processing, filtering, and feature extraction more effective.

EXPERIMENT NO-3

To Extract Features From Given Data Set And Establish Training Data.

(Extract word counts from news articles)

“Extract word counts from news articles using Python” means:

👉 Converting text (news articles) into numbers (word frequencies) so that machine learning models can use it as training data.

Simple Meaning

- Each word = feature
- Count how many times each word appears in an article

👉 Example:

Article:

"India wins match"

Becomes:

India=1, wins=1, match=1

Why we do this?

Machines cannot understand text directly, so we convert text → numerical format (feature extraction).

Code

```
import numpy as np
import pandas as pd
import re #re means regular expression, it extracts numbers or remove
unwanted things from text
from nltk.corpus import stopwords #nltk.corpus is a module in the NLTK
library used to access large collections of text data (corpora).
#NLTK = Natural Language Toolkit (used for NLP tasks)
'''from nltk.corpus import stopwords
print(stopwords.words('english'))
output(short)-['the', 'is', 'in', 'and', 'to', ...]
means-Used to remove unnecessary words'''
from nltk.stem.porter import PorterStemmer
#Stemming = reducing a word to its root/base form by removing suffixes.
```

```

# words = ["playing", "played", "plays", "study"] :>PLAY
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
import nltk
nltk.download('stopwords')
# printing the stopwords in English
print(stopwords.words('english'))
# loading the dataset to a pandas DataFrame
news_dataset=pd.read_csv("C:/Users/Admin/Desktop/AIML_SE/dataset/3
news fake.csv")
# counting the number of missing values in the dataset
news_dataset.isnull().sum()
# replacing the null values with empty string
news_dataset=news_dataset.fillna("")
# merging the author name and news title
news_dataset['content']=news_dataset['author']+' '+news_dataset['title']
# separating the data & label
x=news_dataset.drop(columns='label',axis=1)
y=news_dataset['label']
ps = PorterStemmer()
def stem_text(text):
    words = text.split()          # split sentence into words
    stemmed = [ps.stem(word) for word in words]
    return " ".join(stemmed)      # join back to sentence
news_dataset['stemmed_content'] = news_dataset["content"].apply(stem_text)
print(news_dataset)
#separating the data and label
x=news_dataset['stemmed_content'].values
y=news_dataset['label'].values
# converting the textual data to numerical data
vectorizer = TfidfVectorizer()
vectorizer.fit(x)
x= vectorizer.transform(x)

```

NOTE

now data is ready i.e.we converted text data to numerical means we extracted features and established training data i.e.x values
#now we can train model and test accordingly

EXPERIMENT NO-4

To use PCA for dimensionality reduction

Apply PCA on air pollution data

- **PCA (Principal Component Analysis)** is used to **reduce the number of features** while keeping maximum information.
- PCA transforms your original variables (e.g., temperature, precipitation) into a smaller set of principal components (PCs) that:
- Capture most of the variance in the data, Remove redundancy (correlated variables)
- Step-by-Step Process

1. Collect Climate Data

Typical dataset might include:

Temperature (daily/monthly), Rainfall, Humidity, Wind speed, CO₂ levels

2. Standardize the Data

Climate variables have different units, so standardization is crucial:

This ensures all variables contribute equally.

3. Compute Covariance Matrix

This captures how variables relate to each other.

4. Apply PCA (Eigen Decomposition)

PCA finds:

Eigenvectors → directions (principal components)

Eigenvalues → importance (variance explained)

5. Select Top Components

Keep only the first few PCs that explain most variance (e.g., 90–95%).

6. Transform Data

Project original data onto selected PCs → reduced dataset.

- **NOTE**

High variance → more information

Low variance → less useful

👉 PCA tries to:

Keep directions with maximum variance

Code

1 IMPROT LIBRARIES

import pandas as pd

from sklearn.preprocessing import StandardScaler

'''StandardScaler is a preprocessing technique in machine learning used to standardize features by removing the mean and scaling to unit variance.'''

from sklearn.decomposition import PCA

2 LOAD AIR POLLUTON DATA

**df=pd.read_csv("C:/Users/Admin/Desktop/AIML_SE/dataset/4
Indian_Climate_Dataset_2024_2025.csv",encoding='latin1')**

Step 3: Inspect data

print(df.head())

print(df.columns)

Step 4: Select minimum required features

features=['Temperature_Avg (°C)','Humidity (%)','Rainfall (mm)','Wind_Speed (km/h)']

X = df[features]

X.head()

Step 5: Handle missing values

X = X.dropna()

Step 6: Standardize data

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)

Step 7: Apply PCA (full first)

```
pca = PCA() #here we can give either components or information like 95% or 90%  
etc
```

```
X_pca = pca.fit_transform(X_scaled)
```

```
# Step 8: Explained variance
```

```
explained_variance = pca.explained_variance_ratio_
```

```
#Explained variance ratio = importance of each component
```

```
print("Explained Variance Ratio:", explained_variance)
```

```
# Step 9: Cumulative variance plot
```

```
plt.figure()
```

```
plt.plot(np.cumsum(explained_variance), marker='o')
```

```
plt.xlabel("Number of Components")
```

```
plt.ylabel("Cumulative Variance")
```

```
plt.title("PCA - Climate Data")
```

```
plt.grid()
```

```
plt.show()
```

```
# Step 10: Reduce to 2 components
```

```
pca2 = PCA(n_components=2)
```

```
X_reduced = pca2.fit_transform(X_scaled)
```

```
# Step 11: Create DataFrame
```

```
pca_df = pd.DataFrame(X_reduced, columns=['PC1', 'PC2'])
```

```
print(pca_df.head())
```

Final Conclusion

- Reduced 4 features → 2 components
- Identified:
 - Hot & dry regions
 - Humid & rainy regions

- PCA helps in:
 - ✓ Trend detection
 - ✓ Visualization
 - ✓ Feature reduction

Experiment 05

TO classify features/ To develop classification model and evaluate its performance (any one classifier)

- We will do Bank Customer Classification using Credit Scoring using (Logistic Regression)
- We will classify bank customers as Good Credit (1) or Bad Credit (0) based on their financial and personal features.

• What does this mean?

Classify features”

Use input variables (features) like:

- Age, Income, Credit amount, Loan duration
- These features help in making a decision (classification).
- In simple words: features will help predicting the output good (1) or bad (0)

• Develop a classification model

This means:

- Build a machine learning model that can classify data into categories.

Example (our case):

- Classify customers into:
- Good Credit (1)

- Bad Credit (0)

Common classifiers:

- Logistic Regression (simple & best for exams)
- Decision Tree
- Random Forest
- SVM

Code

Step 1: Import libraries

```
import pandas as pd
```

```
import numpy as np
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

Step 2: Load dataset

```
url = "https://raw.githubusercontent.com/selva86/datasets/master/GermanCredit.csv"
```

```
data = pd.read_csv(url)
```

Step 3: Display dataset

```
data.head()
```

```
print(data.shape)
```

#see all columns

```
print(data.columns.tolist())
```

#see datatypes of features

```
print(data.dtypes)
```

#to see how many isnull rows are there in column

```
print(data['credit_risk'].isnull().sum())
```

#how to see unique variables in the class

```
print(data['status'].nunique())
```

#how to see names of variables

```
data['credit_risk'].value_counts()
```

Step 4: Target variable

'credit_risk' → good (1), bad (0)

```
data['credit_risk'] = data['credit_risk'].replace({1:'good', 0:'bad'})
```

Step 5: Convert categorical variables to numeric

```
data = pd.get_dummies(data, drop_first=True)
```

data.shape #this is a output with drop_first ans will be (1000,49) if without drop first it show (1000,63)

Step 6: Split features and target

```
X = data.drop('credit_risk_good', axis=1)
```

```
y= data['credit_risk_good']
```

Step 7: Train-test split

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Step 8: Feature scaling

```
scaler = StandardScaler()
```

```
X_train = scaler.fit_transform(X_train)
```

```
X_test = scaler.transform(X_test)
```

Step 9: Model training (Logistic Regression)

```
model = LogisticRegression()
model.fit(X_train, y_train)
```

Step 10: Prediction

```
y_pred = model.predict(X_test)
```

Step 11: Evaluation

```
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

RESULTS

Accuracy: 0.805

Confusion Matrix:

```
[[ 35  24]
 [ 15 126]]
```

Classification Report:

	precision	recall	f1-score	support
False	0.70	0.59	0.64	59
True	0.84	0.89	0.87	141
accuracy		0.81		200
macro avg	0.77	0.74	0.75	200
weighted avg	0.80	0.81	0.80	200

conclusion:

Classification of features refers to using input variables such as age, income, and credit amount to predict a categorical outcome. A classification model like Logistic Regression is developed to classify customers into categories such as good or bad credit risk. The model performance is evaluated using metrics like accuracy, precision, recall, F1-score, and confusion matrix.

EXPERIMENT 06
To develop regression model and evaluate its performance (any one algorithm). (Do house house price prediction)

◆ What is Regression?

Regression is a supervised machine learning technique used to predict continuous values (e.g., price, salary, temperature).

👉 Example:

- **Predict house price based on size**
- **Predict salary based on experience**

◆ Steps to Build Model

- 1. Import dataset**
- 2. Split into training & testing**
- 3. Train model**
- 4. Predict output**
- 5. Evaluate performance**

Evaluation matrix:

Mean Absolute Error (MAE): The average of the absolute differences between predictions and actual values.

CODE

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, r2_score
df=pd.read_csv("C:/Users/Admin/Desktop/AIML_SE/dataset/6 House_Price_Data.csv")
df.head()
#Convert categorical column (location) into numeric
df = pd.get_dummies(df, columns=["location",'propertytype'], drop_first=True)
#Features and target
X = df.drop("totalprice", axis=1)
y = df["totalprice"]
#Train-Test Split
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=40)
model = LinearRegression()
#Train Model
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("MAE:", mean_absolute_error(y_test, y_pred))
print("R2 Score:", r2_score(y_test, y_pred))
```

output-

by linear regression

MAE: 6857767.179619476

R2 Score: 0.21473004798038098

by Random forest

MAE: 5111971.817736283

R2 Score: 0.3724479658658829

Conclusion:- House price prediction in India uses regression models to estimate property values based on features like area, BHK, and location. Linear regression provides a baseline model for prediction. Model performance is evaluated using MAE and R^2 score. Advanced models like Random Forest improve prediction accuracy.

Experiment

EXPT-7

Reinforcement Learning Based Predictive Maintenance Project Report

1. Introduction

In modern mechanical industries, machine failure leads to high downtime and financial loss. Traditional maintenance strategies include:

- Reactive Maintenance (after failure)
- Preventive Maintenance (fixed schedule)

These methods are inefficient because they do not adapt to real-time machine conditions. This project uses Reinforcement Learning (RL) to develop an intelligent system that decides when to repair a machine to minimize cost and avoid breakdown.

2. Objectives

- Develop an RL-based decision-making model
- Minimize maintenance cost
- Avoid unexpected machine failure
- Improve machine life and efficiency

3. Problem Statement

Design an intelligent agent that observes machine condition and decides:

- Continue operation
- Perform maintenance (repair)

4. Reinforcement Learning Overview

Reinforcement Learning is a learning technique where an agent interacts with an environment and learns via rewards.

Key Components:

- Agent → Decision maker
- Environment → Machine system
- State (S) → Machine condition
- Action (A) → Repair or Continue
- Reward (R) → Cost or benefit

5. System Model (MDP)

This problem is modeled as a Markov Decision Process (MDP).

States:

- $0 \rightarrow$ Good
- $1 \rightarrow$ Degrading
- $2 \rightarrow$ Critical

Actions:

- $0 \rightarrow$ Continue
- $1 \rightarrow$ Repair

Rewards

- $5 \rightarrow$ Smooth operation
- $10 \rightarrow$ Repair cost
- $50 \rightarrow$ Failure

6. Mathematical Model

Q-Learning Update Rule:

$$Q(s,a) = Q(s,a) + \alpha [R + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

Where:

- $\alpha \rightarrow$ Learning rate
- $\gamma \rightarrow$ Discount factor

7. Algorithm Flowchart

Start



Initialize Q-table



Observe state



Choose action (ϵ -greedy)



Perform action



Receive reward



Update Q-value

↓

Repeat until convergence

↓

End

8. System Architecture Diagram

+-----+

| Machine System |

+-----+

↓ State

+-----+

| RL Agent |

| (Q-Learning Model) |

+-----+

↓ Action ↑ Reward

+-----+

| Maintenance Decision|

+-----+

Note-

1. **Initializing a Q-table** means creating and setting the initial values for all state–action pairs before learning starts. **Generally $Q(s,a)$ -state/action**
 $Q = \text{np.zeros}((\text{state}, \text{action}))$
2. **Alpha means (learning rate)**=Controls how much new information overrides old information
 $Q(s,a)=Q(s,a)+\alpha[r+\gamma\max_{a'}Q(s',a')-Q(s,a)]$
Interpretation:
 - $\alpha = 0.1 \rightarrow$ slow learning (more stable)
 - $\alpha = 1 \rightarrow$ only latest reward matters
3. **Gamma (γ) = 0.9 $\rightarrow$ Discount Factor**
Controls importance of future rewards
Range: 0 to 1
Interpretation:
 - $\gamma = 0 \rightarrow$ only immediate reward matters
 - $\gamma = 0.9 \rightarrow$ future rewards are highly valued
 - $\gamma = 1 \rightarrow$ long-term focus
4. **Epsilon (ϵ) = 0.2 $\rightarrow$ Exploration Rate**
Used in ϵ -greedy policy
Controls exploration vs exploitation
Interpretation:
 - $\epsilon = 0 \rightarrow$ always exploit (no exploration)
 - $\epsilon = 1 \rightarrow$ fully random actions

$\epsilon = 0.2 \rightarrow 20\%$ exploration, 80% exploitation

CODE

```
import numpy as np
import random
Q = np.zeros((3, 2))
alpha = 0.1
gamma = 0.9
epsilon = 0.2
def next_state(state, action):
    if action == 1:
        return 0
    return min(state + 1, 2)
def reward(state, action):
    if action == 1:
        return -10
    elif state == 2:
        return -50
    return 5
for episode in range(1000):
    state = random.randint(0, 2)
    for step in range(20):
        if random.random() < epsilon:
            action = random.randint(0, 1)
        else:
            action = np.argmax(Q[state])
        ns = next_state(state, action)
        r = reward(state, action)
        Q[state, action] += alpha * (
            r + gamma * np.max(Q[ns]) - Q[state, action] )
        state = ns

print("Q-table:\n", Q)
```

OUTPUT

Q-table:

```
[[ 5.16605166 -5.35055351]
```

[0.18450185 -5.35055351]

[-54.81549815 -5.35055351]]

◆ Interpretation (Very Important)

- Continue → +5.17 (good)
- Maintenance → -5.35 (loss)

Correct: Don't maintain a healthy machine

State 1 (Degrading)

- Continue → +0.18 (slightly good)
- Maintenance → -5.35 (costly)

Agent says:

“Still okay to run, no need for maintenance yet”

State 2 (Critical)

- Continue → -54.82 (very bad)
- Maintenance → -5.35 (less bad)

Strong learning:

“Avoid continuing—do maintenance!”

5. Results and Discussion

- The agent learns to repair only when condition is critical
- Avoids unnecessary maintenance
- Reduces failure penalties

Sample Insight:

- Good state → Continue
- Critical state → Repair

Expt 8-Crack Detection in concrete Materials using CNN

Concept

Instead of manual inspection, a CNN automatically learns features like:

- edges
- textures
- irregular patterns

to classify images as:

☞ Crack

☞ No Crack

System Architecture

Input Image → Preprocessing → CNN Model → Output (Crack / No Crack)

Steps:

1. Image collection
2. Image preprocessing
3. Model training
4. Prediction

Dataset

You can use:

- Concrete Crack Images Dataset (Kaggle)
- Surface defect dataset

Context

Concrete surface cracks are major defect in civil structures. Building Inspection which is done for the evaluation of rigidity and tensile strength of the building. Crack detection plays a major role in the building inspection, finding the cracks and determining the building health.

Code

#1. Import Libraries

```
import tensorflow as tf  
from tensorflow.keras import layers, models  
import matplotlib.pyplot as plt
```

#2. Load Dataset

```
from tensorflow.keras.datasets import mnist  
(X_train, y_train), (X_test, y_test) = mnist.load_data()
```

#Step 3: Understand Data

```
print("Training shape:", X_train.shape)  
print("Testing shape:", X_test.shape)
```

#Step 4: Normalize Data

```
X_train = X_train / 255.0  
X_test = X_test / 255.0
```

#Step 5: Reshape for CNN

#CNN expects 4D input → (samples, height, width, channels)

```
X_train = X_train.reshape(-1, 28, 28, 1)  
X_test = X_test.reshape(-1, 28, 28, 1)
```

#Using 60000 works only if the dataset size is exactly 60000, while -1 is preferred because it automatically adjusts the number of samples.

```
model = models.Sequential()
```

Convolution Layer

```
model.add(layers.Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)))
```

Pooling Layer

```
model.add(layers.MaxPooling2D((2,2)))
```

Second Conv Layer

```
model.add(layers.Conv2D(64, (3,3), activation='relu'))
```

Second Pooling

```
model.add(layers.MaxPooling2D((2,2)))
    # Flatten
model.add(layers.Flatten())

    # Fully Connected Layer
model.add(layers.Dense(64, activation='relu'))

    # Output Layer
model.add(layers.Dense(10, activation='softmax'))
    #Step 7: Compile Model
model.compile(optimizer='adam',
loss='sparse_categorical_crossentropy',
metrics=['accuracy'])
    #Step 8: Train Model
history = model.fit(X_train, y_train, epochs=5, validation_data=(X_test,
y_test))
    #Step 9: Evaluate Model
test_loss, test_acc = model.evaluate(X_test, y_test)
print("Test Accuracy:", test_acc)
    #Step 10: Predict Output
predictions = model.predict(X_test)

import numpy as np
print("Predicted:", np.argmax(predictions[0]))
print("Actual:", y_test[0])
```